

submission

Tech Challenge - Fases 1 e 2 - Documento de entrega (back-end Oficina)

Identificação do grupo

- **Nome do grupo:** Oficina Turbo
- **Código/identificador interno:** Grupo 106

Participantes (nome + Discord)

- **Felipe Ricarte Magalhães** - Discord: **@felipe.ricarte**

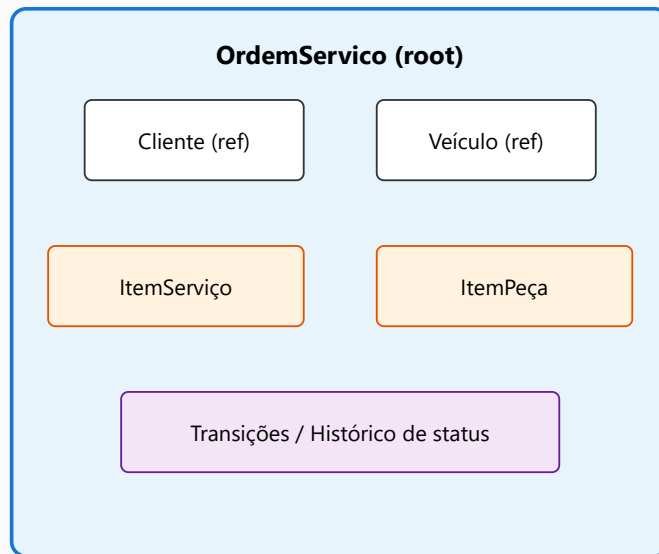
Links do projeto

- **Repositório (privado):** <https://github.com/ricartefelipe/oficina-springboot-mvp>
 - **Acesso ao avaliador SOAT:** utilizador de organização **soat-architecture** com permissão de leitura (ou conforme o enunciado). Passos: [Convidar o utilizador no GitHub](#).
- **Documento para o PDF do portal (Fase 2):** [entrega-portal-fase2.md](#) - *estrutura alinhada ao enunciário (repo, arquitetura, vídeo, Swagger)*.
- **Swagger (local):** <http://localhost:8080/api/swagger-ui/index.html>
- **Vídeo demonstrativo (≤ 15 min):** o URL público (YouTube ou Vimeo) será o mesmo indicado na tabela **Links rápidos (APIs e vídeo)** do [README](#) quando estiver publicado.
- **PDF deste documento (gerado):** `submission.pdf` (na mesma pasta; ver [Conversão para PDF](#)).

Diagramas de arquitetura (DDD)

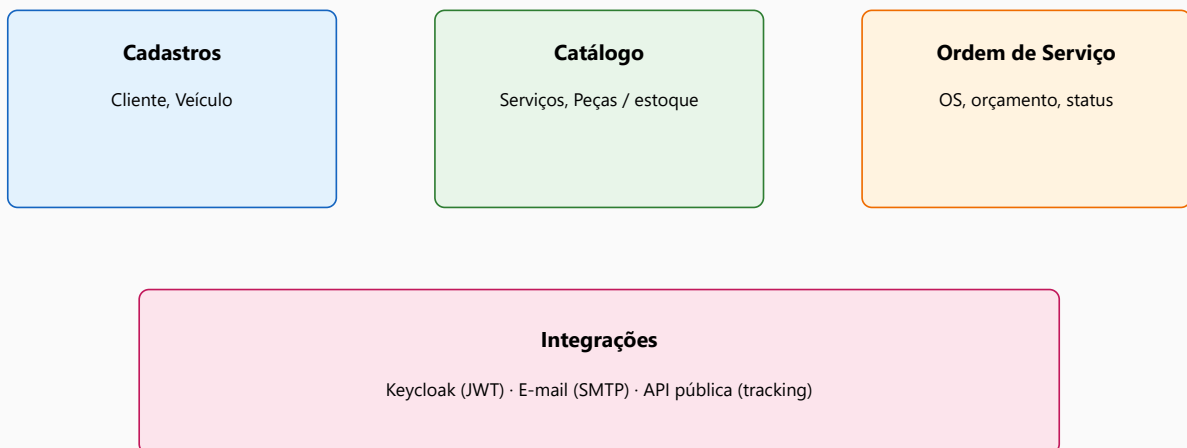
Artefatos de texto: [índice DDD](#), [Domain Storytelling](#), [Dicionário linguagem ubíqua](#).

Agregado: Ordem de Serviço



Invariantes no domínio; API e persistência via adaptadores

Event Storming (resumo) — Oficina



Ver também event-storming.md e diagramas.md (Mermaid)

Diagrama - event storming lousa (elementos C/A/E/P/R)

Checklist entrega Fase 2

O que só podes fazer na tua conta (GitHub, portal da disciplina, gravação do vídeo). Um assistente automatizado não consegue substituir estes passos.

1. Convidar `soat-architecture` no repositório

1. Abre <https://github.com/ricartefelipe/oficina-springboot-mvp>
2. **Settings** → **Collaborators and teams** (ou **Manage access**)
3. **Add people / Invite a collaborator**
4. Pesquisa `soat-architecture` (utilizador ou equipa indicada no enunciado) e envia convite com permissão adequada (normalmente **Read** para revisão).
5. Confirma no e-mail/GitHub que o convite foi aceite (se aplicável).

2. PDF no portal da disciplina

1. Gera o PDF deste documento (secção [Conversão para PDF](#)).
2. Inclui no PDF (ou anexos) os **diagramas DDD** referidos em [docs/ddd/diagramas.md](#) e os SVG em [docs/ddd/diagrams/](#) se o enunciado pedir evidência visual.
3. Faz o upload no **portal SOAT** no prazo indicado pelo professor (o assistente não tem acesso ao portal).

3. Vídeo (≤ 15 minutos)

Roteiro e tópicos obrigatórios: [docs/video-script.md](#). Após publicar no **YouTube** ou **Vimeo**, atualiza o URL no [README](#) (tabela **Links rápidos**); o documento de portal [entrega-portal-fase2.md](#) deve refletir o mesmo link na respetiva secção.

Resumo do entregável

Fase 1 (MVP)

- Reforço pós-feedback do docente: [Domain Storytelling](#), [Dicionário de linguagem ubíqua](#), Event Storming com [figuras SVG](#) (incl. lousa com elementos e agregados), além do texto já existente.
- CRUDs administrativos: clientes, veículos, serviços, peças/insumos (estoque)
- Ordens de serviço: criação, orçamento automático, transições de estado, listagem e detalhe
- Consulta pública por `trackingCode` e aprovação de orçamento com validações (CPF/CNPJ)
- Segurança: JWT (Keycloak, role ADMIN) nos endpoints admin
- Métrica: tempo médio de execução (EM_EXECUCAO → FINALIZADA)
- Dockerfile + Docker Compose
- Testes (unitários e integração) + JaCoCo
- Documentação DDD (Event Storming, diagramas, linguagem ubíqua) + roteiro de vídeo

Fase 2 (resiliência e escalabilidade)

- Arquitetura em evolução (hexagonal onde aplicável), testes nos fluxos críticos

- **Kubernetes:** manifestos em `/k8s` (Deployment, Service, HPA, probes, etc.)
- **IaC:** Terraform em `/infra` (rede AWS; **RDS PostgreSQL opcional** via `enable_rds`)
- **CI/CD:** GitHub Actions (Maven, Terraform validate, imagem **GHCR**; workflows manuais para deploy em cluster e Terraform na AWS)
- Diagramas DDD versionados (SVG em `docs/ddd/diagrams/`)

Relatório de vulnerabilidades (scan)

- **Arquivo do relatório:** `docs/security/vulnerability-report.md`
- **Evidências geradas pelo scan:** `build/security/*`
- **Resumo (valores de exemplo):**
 - CRITICAL: 0
 - HIGH: 0
 - MEDIUM: 2
 - LOW: 6
- **Plano de mitigação:** descrito no relatório (upgrade de dependências, ajustes de configuração e revisão contínua no pipeline).

Como executar localmente (para o avaliador)

Pré-requisito: Docker + Docker Compose v2

1. Subir o ambiente:

```
docker compose up --build
```

2. Obter token ADMIN (Keycloak):

```
export TOKEN="$(./scripts/get-admin-token.sh)"
```

3. Validar endpoints (exemplo):

```
curl -i http://localhost:8080/api/admin/clientes \
-H "Authorization: Bearer ${TOKEN}"
```

4. Rodar testes e cobertura:

```
mvn verify
```

5. Rodar scan de vulnerabilidades e gerar evidências:

```
./scripts/security/run-security-scans.sh
```

Conversão para PDF (offline)

Recomendado (Windows, PDF com diagramas embutidos): na raiz do repositório:

```
.\scripts\delivery\md-to-pdf-edge.ps1 -InputMd "docs\delivery\submission.md"
```

Gera docs/delivery/submission.pdf (e um .html intermédio, ignorado pelo Git).

Com [Pandoc](#) só (sem diagramas rasterizados como acima):

```
pandoc docs/delivery/submission.md -o docs/delivery/submission.pdf
```

Sem Pandoc no Windows: `winget install JohnMacFarlane.Pandoc`, ou abrir este .md num editor com pré-visualização Markdown e usar **Imprimir** → **Guardar como PDF**.